the moving average of a sliding window of data, such as the hourly average sales during a cyber-Monday sale. Metric and bucket aggregations won't allow us to chain aggregations.

Elasticsearch provides a third set of aggregations called *pipeline aggregations* that permit aggregation chaining. These aggregations work on the output of other aggregations rather than individual documents or fields of documents. That is, we create a pipeline aggregation by passing the output of a bucket or metric aggregation. Before we get our hands dirty, let's look at the types of pipelines, their syntax, and other details.

### 13.5.1 Pipeline aggregation types

Broadly speaking, we can group pipeline aggregations into two types: *parent* and *sibling*. As explained in section 13.4, parent pipeline aggregations work on the input from the child aggregation to produce new buckets or new aggregations, which are then added to existing buckets. Sibling pipeline aggregations produce a new aggregation at the same level.

### 13.5.2 Sample data

We look at the parent and sibling aggregation types in detail as we execute some examples in this section. We use the `coffee_sales` data set for running pipeline aggregations. Follow the usual process of indexing data using the `_bulk` API, as in the following listing. You can fetch the sample data from the book's repository on GitHub (http://mng.bz/6D45) or the book's website.

**Listing 13.21 Indexing data using the `_bulk` API**

```
PUT coffee_sales/_bulk
{"index":{"_id":"1"}}
{"date":"2022-09-01","sales":{"cappuccino":23,"latte":12,"americano":9,
➥ "tea":7},"price":{"cappuccino":2.50,"latte":2.40,"americano":2.10,
➥ "tea":1.50}}
{"index":{"_id":"2"}}
{ "date":"2022-09-02","sales":{"cappuccino":40,"latte":16,"americano":19,
➥ "tea":15},"price":{"cappuccino":2.50,"latte":2.40,"americano":2.10,
➥ "tea":1.50}}
```

Executing this query indexes two sales documents into our `coffee_sales` index. The next step is to create pipeline aggregations to help us understand them in detail.

### 13.5.3 Syntax for pipeline aggregations

As mentioned earlier, pipeline aggregations work on the input from other aggregations. So, when declaring a pipeline, we must provide a reference to those metric or bucket aggregations. For our example, we can set a reference as `buckets_path`, which consists of aggregation names with an appropriate separator in the query. The `buckets_path` variable is a mechanism to identify the input to the pipeline query.